

Verdict

- **Open-source variant:** Approve with changes
- **LocalStack variant:** Reject

Overall Recommendation: Proceed exclusively with the Open-Source variant. Discard the LocalStack variant entirely due to licensing risks and critical pedagogical flaws in teaching security without policy enforcement.

Executive assessment

Strongest qualities: The program brilliantly captures the reality of the AI Forward Deployed Engineer role. By structuring the curriculum around six sequential enterprise engagements—starting with diagnosing an existing, messy system rather than a greenfield project—it perfectly mirrors field-delivery reality. The integration of technical execution with stakeholder management, scope negotiation, and "dual-room" (technical vs. executive) defenses ensures learners graduate as true FDEs, not just isolated engineers. The strict local-first boundary for P1–P5 and the bounded 14-day AWS capstone in P6 cleverly solves the traditional bootcamp runaway-cost problem.

Largest risks: The LocalStack variant introduces significant licensing uncertainty (LocalStack Hobby ToS) and teaches "security theater" by exercising IAM/KMS APIs that do not actually enforce policies locally. Across both variants, running a complete multi-container stack (PostgreSQL, OpenSearch, MinIO/RabbitMQ, observability tools, and local LLM inference via Ollama) locally will severely overwhelm 16GB RAM machines. If memory limits and tiny LLMs (<3B parameters) are not strictly enforced in the provided scaffolds, students will lose dozens of hidden hours to OOM (Out of Memory) crashes.

Scorecard

Open-source variant

- **FDE role alignment:** 5
- **Field-delivery coverage:** 5
- **Technical depth:** 5
- **Project-first progression:** 5
- **Workload feasibility:** 4
- **Assessment scalability:** 5
- **Local development feasibility:** 4
- **AWS budget feasibility:** 5
- **Cloud-transition realism:** 4
- **Job readiness:** 5

LocalStack variant

- **FDE role alignment:** 5
- **Field-delivery coverage:** 5
- **Technical depth:** 3
- **Project-first progression:** 5
- **Workload feasibility:** 3
- **Assessment scalability:** 4
- **Local development feasibility:** 3
- **AWS budget feasibility:** 5
- **Cloud-transition realism:** 3
- **Job readiness:** 4

Explanation of scores below 4 (LocalStack Variant): - **Technical depth (3):** Teaching AWS IAM and KMS without actual policy enforcement in the Hobby tier actively teaches students to ignore security failures or write policies that are never validated. - **Workload feasibility (3):** LocalStack often has subtle API mismatches compared to real AWS. Students will waste hours debugging LocalStack emulator quirks rather than learning application logic. - **Local development feasibility (3):** Running the heavy LocalStack Java/Python services alongside an observability stack and local LLMs will heavily strain student hardware. - **Cloud-transition realism (3):** Because the Hobby tier lacks genuine IAM/RBAC enforcement, the transition to real AWS in P6 will cause applications to crash due to missing permissions, frustrating students who falsely believed their local code was production-ready.

Blocking findings

1. Security Theater via Unenforced IAM

- **File:** `fde-focused-6-projects-localstack.md`
- **Project and task:** Project 4, Task 4.3
- **Problem:** Using LocalStack Hobby to teach IAM and KMS when it does not actually enforce IAM policies.
- **Why it matters:** It teaches security theater. Students will write invalid IAM policies or skip them entirely, and their local tests will still pass. When deployed to AWS in Project 6, the system will immediately break.
- **Concrete correction:** Reject the LocalStack variant entirely to avoid this.
- **Estimated hours impact:** +0 (Rejecting variant saves wasted authoring hours).

2. Hardware Resource Constraints (OOM Risks)

- **File:** `fde-focused-6-projects-opensource.md`
- **Project and task:** Program-wide (Implied across P2–P5)
- **Problem:** Running PostgreSQL, OpenSearch, MinIO, Keycloak, Prometheus, Phoenix, and Ollama simultaneously will crash standard 16GB RAM machines.
- **Why it matters:** Breaks the "Local-First Development" constraint for the target hardware baseline defined in `AGENTS.md`.
- **Concrete correction:** Add an explicit constraint to the program requirements: "Starter scaffolds must hardcode Docker Compose memory limits (`mem_limit`) and mandate sub-3B parameter models (e.g., Qwen 2.5 1.5B or Llama 3.2 3B) for local inference to guarantee 16GB RAM compatibility."
- **Estimated hours impact:** +0 for students, but requires careful infrastructure engineering by curriculum authors.

3. P6 Cloud Transition Requires Strict Architecture

- **File:** `fde-focused-6-projects-opensource.md`
- **Project and task:** Project 6, Task 6.2
- **Problem:** "Replace local infrastructure adapters with supplied AWS bindings" is estimated at 18 hours.
- **Why it matters:** If the P1–P5 application code is tightly coupled to MinIO or RabbitMQ, swapping to S3/SQS will require a massive rewrite, completely blowing the 18-hour estimate and failing the capstone.
- **Concrete correction:** Add an explicit requirement to Task 1.2 and Task 2.3 that the provided codebase strictly enforces a Dependency Inversion / Ports & Adapters architecture.
- **Estimated hours impact:** Saves 20+ hours of student refactoring in P6.

Important findings

1. Orphaned Fine-Tuning Task

- **File:** `fde-focused-6-projects-opensource.md`
- **Project and task:** Project 5, Task 5.3
- **Problem:** "PEFT/LoRA preparation is optional... Any AWS execution is deferred to Project 6." However, Project 6 has no task or infrastructure pathway for actually executing or deploying fine-tuning.
- **Why it matters:** Creates an orphaned optional task that students prepare for but can never execute.
- **Concrete correction:** Remove the PEFT/LoRA mention entirely. It distracts from the core FDE competencies of agent bounding and evaluation.
- **Estimated hours impact:** -2 hours.

2. Misaligned Diagnostic Hours

- **File:** `fde-focused-6-projects-opensource.md`
- **Project and task:** Project 1, Tasks 1.3, 1.4, 1.5
- **Problem:** 18 hours are allocated for a single "diagnose and repair" technical task (1.3), leaving only 10 and 8 hours for the critical communication and handoff tasks.
- **Why it matters:** The 18-hour estimate is disproportionate if the bug is seeded cleanly. FDEs need more time practicing the business synthesis and defense.
- **Concrete correction:** Reduce Task 1.3 to 14 hours. Shift 2 hours to Task 1.4 (Problem framing) and 2 hours to Task 1.5 (Diagnostic handoff).
- **Estimated hours impact:** 0 (redistributed).

Variant comparison

- **Outcomes that are genuinely equivalent:** Both variants teach the exact same FDE lifecycle—stakeholder management, scope negotiation, hybrid RAG implementation, agent bounding, and evaluation metrics.
- **Outcomes or skills that differ unintentionally:** The Open-Source variant teaches genuine RBAC and OIDC via Keycloak. The LocalStack variant teaches AWS IAM syntax but fails to enforce it, leaving a massive skill gap in actual security implementation.
- **Infrastructure that adds complexity without sufficient learning value:** LocalStack adds significant emulator-specific quirks and licensing uncertainty (Hobby tier ToS) without providing true AWS fidelity.
- **Which variant is safer to launch:** The **Open-source variant**. It avoids ToS gray areas and provides a deterministic, true-to-behavior local environment.
- **Which variant better prepares learners for AWS delivery:** Paradoxically, the **Open-source variant**. Because learners are forced to use real, enforced security (Keycloak) and standard APIs (S3 via MinIO), they understand the abstract concepts deeply. When they swap adapters in Project 6, they map solid concepts to AWS, rather than untangling why a LocalStack script passed locally but fails on AWS.

Workload audit

- **Hours per project:** P1 (60), P2 (80), P3 (80), P4 (80), P5 (80), P6 (60).
- **Total task hours:** Exactly matches the 440-hour limit.
- **Likely hidden setup or troubleshooting hours:** Expect +15 hours of unmeasured troubleshooting over the program primarily due to Docker Compose networking issues and OOM (Out of Memory) crashes if local LLMs are too large.
- **Tasks most likely to exceed their estimates:**
- **Task 4.3 (Implement the security boundary - 30 hours):** This is extremely dense. Integrating OIDC, prompt controls, redactors, and CI gates could easily spill into 40+ hours for engineers new to IAM concepts. The provided Keycloak scaffold must be strictly plug-and-play.
- **Task 6.2 (AWS deployment - 18 hours):** Will blow up if the application wasn't built with decoupled adapters.

AWS and GPU audit

- **The \$180 AWS cap:** Highly realistic for a 14-day deployment of standard compute, RDS, and API Gateway, assuming teardown is enforced.
- **14-day deployment window:** Excellent constraint.
- **Temporary endpoint architecture:** Sound and cost-effective.
- **Scheduled GPU allowance:** 4–8 hours of a g4dn.xlarge or g5.xlarge is ~\$2–\$8 per student. Very affordable, and the scheduling mechanism prevents runaway costs. Do not recommend an always-on GPU.
- **Cost Risks & Teardown Contract:** The largest risk is a student abandoning the course mid-capstone. **Recommendation:** Implement an automated, course-managed AWS Nuke script or strict tagging-based Lambda cleanup that forcibly destroys any resource older than 14 days, removing reliance on the student's 6.6 teardown execution.

Project-by-project findings

- **P1 (Diagnostics):** Very strong start. Simulating an FDE stepping into a messy, existing system rather than a greenfield project is brilliant and highly aligned with reality.
- **P2 (Hybrid RAG):** Excellent scope control. Bounding ingestion to a "supplied enterprise document set" avoids the infinite rabbit hole of unstructured data parsing and OCR library selection.
- **P3 (Reliability):** The focus on provider resilience (rate limits, outages, deadlines) is exactly what production LLM applications require.
- **P4 (Security):** Teaching STRIDE and implementing prompt-injection boundaries is highly relevant. However, ensure the OIDC scaffold is flawless to protect the 30-hour budget.
- **P5 (Agents):** Bounding the agent to LangGraph and strictly limiting its authority is the correct enterprise approach. It avoids the trap of building autonomous "do-everything" agents that cannot be safely governed.
- **P6 (Capstone):** Focus on adoption, handback, and two-room defense makes it a true FDE capstone, not just a generic DevOps deployment exercise.

Highest-leverage changes

(In priority order)

1. **Discard the LocalStack variant.** Move forward entirely with the Open-Source variant (`fde-focused-6-projects-opensource.md`) to avoid licensing risks and unenforced security exercises.
2. **Mandate Ports and Adapters architecture.** In P1.2 and P2.3, explicitly state: *"Supplied code scaffolds must enforce a strict dependency inversion (Ports and Adapters) architecture to ensure the cloud transition in Project 6 requires only an adapter swap, not an application rewrite."*
3. **Enforce local RAM constraints.** Add a programmatic constraint: *"All Docker Compose scaffolds must include explicit memory limits (`mem_limit`), and local inference must be restricted to sub-3B parameter models (e.g., Llama 3.2 3B) to guarantee operability on 16GB machines."*
4. **Remove the orphaned fine-tuning task.** In P5.3, delete the bullet point regarding *"PEFT/LoRA preparation is optional..."* to maintain focus and prevent students from preparing for an AWS GPU fine-tuning deployment that doesn't exist in P6.
5. **Add an automated AWS failsafe.** In Task 6.6, specify: *"The curriculum platform will enforce budget safety via an automated, tagging-based cloud cleanup script (e.g., AWS Nuke) that forcefully terminates resources after 14 days."*
6. **Redistribute Project 1 hours.** Adjust P1 estimates: Change Task 1.3 from 18h to 14h. Increase Task 1.4 from 10h to 12h, and Task 1.5 from 8h to 10h.
7. **Ensure the Identity scaffold is plug-and-play.** In Task 4.3, clarify that the Keycloak/OpenBao infrastructure is provided as a pre-configured, instantly runnable container, ensuring students spend time writing application security logic, not configuring Java servers.
8. **Clarify the LLM-as-judge constraint.** In Task 5.4, explicitly state that *"manual LLM-as-judge"* means running the judge locally using the local bounded LLM, preventing accidental massive API bills during CI testing loops.
9. **Remove Kafka/Redpanda mentions.** In Task 3.2, remove the text *"Treat Kafka, Redpanda... as optional extensions"*. To avoid confusion and scope creep, rely solely on RabbitMQ as the open-source async option.
10. **Define asynchronous simulations.** In the Program Delivery Contract, explicitly define *"timed injects"* as the release of new markdown/text requirements via the learning platform at specific commit stages, ensuring no live human roleplay is required for grading.

Final recommendation

Primary program: `fde-focused-6-projects-opensource.md` (Open-source variant). **Fallback:** None. The LocalStack variant should be entirely discarded.

What must change before curriculum production begins: You must explicitly mandate a Ports and Adapters architecture in the provided starter code to make the Project 6 AWS transition survivable within 18 hours. You must establish strict memory constraints (`mem_limit`) and mandate tiny LLMs to support the 16GB local hardware requirement. Finally, remove the orphaned fine-tuning references in P5 to maintain tight focus on the core FDE competencies.